

Kortex — Master Context Document

Save this file. If you need to recreate the project from scratch, share this with Claude and ask it to help you set up the project.

What is Kortex?

An enterprise RAG-powered knowledge management platform built as a Micro-Frontend monorepo.

Users browse curated knowledge articles and query an AI assistant that answers from the organization's knowledge base with source citations and zero hallucinations.

GitHub Repository

- Repository: [Kortex GitHub Repository](#)
- Branch: `master`

Architecture

Turborepo + pnpm monorepo with 4 services:

Service	Directory	Port
Shell (Host MFE)	<code>apps/shell</code>	<code>3000</code>
Content MFE	<code>apps/content-mfe</code>	<code>3001</code>
RAG MFE	<code>apps/rag-mfe</code>	<code>3002</code>
FastAPI RAG Service	<code>services/rag-service</code>	<code>8000</code>

Tech Stack

Frontend

React 18 + TypeScript + Vite

- React 18 + TypeScript + vite
- @module-federation/vite EXACTLY 1.12.3 (no caret, no other version)
- Tailwind CSS v4 Uses:

```
@import "tailwindcss";
```

Not:

```
@tailwind base;
```

- Clerk for authentication (Google + GitHub OAuth)
- lucide-react 1.12.0 (exact version, no caret)
- pdfjs-dist v5 (uses .mjs worker)
- shamefully-hoist=true in ALL MFE .npmrc files

Backend

- Python 3.12 + FastAPI + uvicorn
- LangChain for RAG orchestration
- google-generativeai (needs migration to google.genai)
- Groq SDK
- qdrant-client

External Services

All services require accounts and API keys.

1. Gemini

- Website: [Google AI Studio](#)
- API key → GEMINI_API_KEY
- Embedding model:

```
models/gemini-embedding-001
```

- Embedding dimensions: 3072
- LLM:

```
gemini-2.5-flash
```

2. Groq

- Website: [Groq Console](#)
- API key → GROQ_API_KEY
- Model:

```
llama-3.3-70b-versatile
```

3. Qdrant Cloud

- Website: [Qdrant Cloud](#)
- Create cluster → get URL + API key

Environment variables:

```
QDRANT_URL=  
QDRANT_API_KEY=
```

Configuration:

- Collection name:

```
kortex_knowledge
```

- Vector size: 3072
 - Distance metric: Cosine
 - Collection is auto-created by the RAG service on startup
-

4. Sanity CMS

- Website: [Sanity CMS](#)
- Create project → get project ID + dataset name

Environment variables:

```
SANITY_PROJECT_ID=  
SANITY_DATASET=production  
SANITY_API_TOKEN=
```

Additional details:

- CMS directory:

```
cms/
```

- Schema:

```
article
```

Fields:

- title
- slug
- category
- author
- excerpt
- body
- publishedAt
- tags

5. Clerk

- Website: [Clerk Authentication](#)
- Enable:
 - Google OAuth
 - GitHub OAuth

Environment variable:

```
VITE_CLERK_PUBLISHABLE_KEY=
```

Used in:

- `apps/shell`
- `apps/content-mfe`

6. Vercel

- Website: [Vercel](#)
- Deploy:

```
apps/shell
```

Install command:

```
pnpm install --no-frozen-lockfile
```

Add all environment variables in the Vercel dashboard.

7. Hugging Face Spaces

- Website: [Hugging Face Spaces](#)
- Deploy:

```
services/rag-service
```

as a Docker Space

Add all environment variables as Space secrets.

Environment Variables

services/rag-service/.env

```
GEMINI_API_KEY=  
GROQ_API_KEY=  
QDRANT_URL=  
QDRANT_API_KEY=  
SANITY_PROJECT_ID=  
SANITY_DATASET=production  
SANITY_API_TOKEN=  
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:3002
```

apps/shell/.env

```
VITE_CLERK_PUBLISHABLE_KEY=  
VITE_RAG_SERVICE_URL=http://localhost:8000  
VITE_CONTENT_MFE_URL=http://localhost:3001  
VITE_RAG_MFE_URL=http://localhost:3002
```

apps/content-mfe/.env

```
VITE_CLERK_PUBLISHABLE_KEY=  
VITE_SANITY_PROJECT_ID=  
VITE_SANITY_DATASET=production  
VITE_RAG_SERVICE_URL=http://localhost:8000
```

apps/rag-mfe/.env

```
VITE_RAG_SERVICE_URL=http://localhost:8000
```

Local Development Startup Order

Always start services in this order:

Tab 1 — RAG Service

```
cd services/rag-service
source .venv/bin/activate
uvicorn main:app --reload --port 8000
```

Tab 2 — Content MFE

```
cd /path/to/Kortex
pnpm --filter @kortex/content-mfe dev
```

Tab 3 — RAG MFE

```
pnpm --filter @kortex/rag-mfe dev
```

Tab 4 — Shell

```
pnpm --filter @kortex/shell dev
```

Then open:

```
http://localhost:3000
```

Python Setup (RAG Service)

```
cd services/rag-service

python3 -m venv .venv

source .venv/bin/activate

pip install -r requirements.txt --break-system-packages
```

Critical Technical Constants

- `@module-federation/vite` must be EXACTLY `1.12.3`
- All MFEs require:

```
shamefully-hoist=true
```

in `.npmrc`

- Embedding model:

```
models/gemini-embedding-001
```

- Embedding dimensions:

```
3072
```

- Qdrant collection:

```
kortex_knowledge
```

- Tailwind CSS v4 must use:

```
@import "tailwindcss";
```

- Use:

```
pnpm install --ignore-workspace
```

for Sanity Studio and conflicting apps

- Recreate env files with:

```
cp .env.example .env
```

Key Files Reference

Purpose	File
Module federation config	<code>vite.config.ts</code> in each app
RAG pipeline	<code>services/rag-service/app/rag_pipeline.py</code>
Vector store	<code>services/rag-service/app/vector_store.py</code>
Ingestion endpoint	<code>services/rag-service/app/main.py</code>
Shell routing	<code>apps/shell/src/App.tsx</code>
Content articles	<code>apps/content-mfe/src/components/ArticleList.tsx</code>
RAG chat	<code>apps/rag-mfe/src/components/ChatWindow.tsx</code>
Upload modal	<code>apps/rag-mfe/src/components/UploadPanel.tsx</code>
Source card	<code>apps/rag-mfe/src/components/SourceCard.tsx</code>

Sanity Content Structure

Each article contains:

- `title` (string)
- `slug` (slug)
- `category` (string)
- `author` (string)
- `excerpt` (text)
- `body` (portable text)
- `publishedAt` (datetime)
- `tags` (array of strings)

After adding articles in Sanity Studio:

- Run the ingestion webhook, or
- Restart the RAG service

This re-embeds documents into Qdrant.

Qdrant Point Structure

Each point payload contains:

Field	Description
article_id	UUID for Sanity or dynamic-{hash} for uploads
title	Article title
slug	Slug
category	Category
author	Author
excerpt	Truncated preview
content	Full document text
chunk_index	Chunk number within document
chunk_text	Embedded text chunk
published_at	ISO datetime
source	"dynamic" for uploads

Features Implemented

- 1. Knowledge Hub — browse Sanity articles with category filters
- 2. AI Assistant — streaming RAG chat with source citations
- 3. Add Knowledge — dynamic upload (.txt , .md , .pdf , pasted text)
- 4. Multi-provider AI — Gemini 2.5 Flash ↔ Groq Llama 3.3 toggle
- 5. Clerk authentication — protected routes with Google/GitHub OAuth
- 6. Uploaded Documents section — uploaded knowledge cards
- 7. Dynamic category filters — generated from all sources
- 8. Source card modals — full document preview
- 9. Uploaded document full-page view

Evaluation

RAGAS

Directory:

Dataset:

- 5 QA pairs
- 6 articles

Scores:

Metric	Score
Faithfulness	1.0
Context Recall	0.9
Answer Relevancy	0.517
Context Precision	0.200

k6 Load Testing

Directory:

Scenarios:

- Baseline
- Concurrent
- Stress

Results:

Scenario	Result
Baseline	Stable, 0% error
Concurrent (10 VUs)	Gemini free-tier rate limiting
Stress (20 VUs)	79% error rate due to rate limits

Dissertation Details

Field	Value
Course	SS ZG628T Dissertation
Program	BITS Pilani M.Tech Software Engineering
Student ID	2024TM93147
Project	Kortex – Enterprise Knowledge Management with RAG

Recreate Project Prompt

Use this prompt with Claude:

```
I have a Kortex project on GitHub at https://github.com/UtkarshK95/Kortex.  
  
Here is the master context document:  
[paste this file]  
  
I have set up fresh accounts for [list services].  
  
Help me set up the local development environment from scratch.
```